

Introductory Econometrics with Recitation: TA Session Week 4

Danny Po-Hsien Kang (康柏賢)

March 17, 2025

Today's agenda

- 1 Data Visualization
 - `geom_histogram()`
 - `facet_wrap()`
 - `geom_bar()`
 - `labs()`
 - `factor()`
 - `as.Date()`
 - `geom_line()`
 - `geom_vline()`
 - `geom_hline()`
 - `ggsave()`

About the R Quiz (Quiz 2)

- Quiz 2 will be held on March 24.
- The R Quiz will cover Week 1-4 materials in the TA Session (all slides included).
- All of the problems will be multiple-choice questions.
 - 15 questions, 2 points each.
- The questions will be related to TA Session practices.
- You can find R quizzes last semester through the link in TA Session's Website.
- The Quiz is a written exam, so no laptop is needed.

Today's Dataset

- Today we will use the following csv files from the OpenIntro Website:
 - `loan_full_schema.csv`: The dataset represents thousands of loans made through the Lending Club platform.
 - `ames.csv`: The dataset includes the housing prices in Ames, Iowa, from 2006 to 2010.
- Please import the following datasets into RStudio.

```
housing <- read.csv("data/ames.csv")  
loan <- read.csv("loans_full_schema.csv")
```

Histogram: `geom_histogram()`

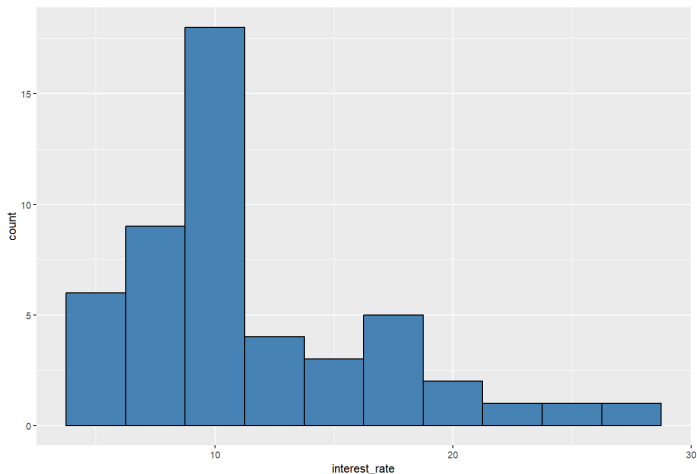


Figure: Histogram of interest rates.

Histogram: geom_histogram()

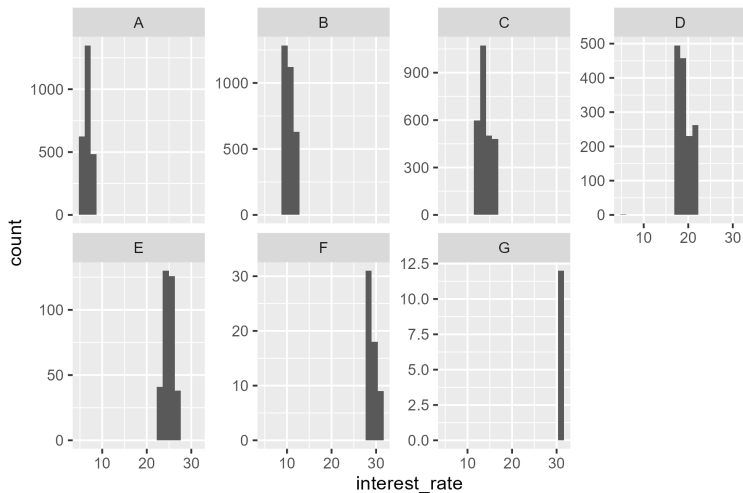
- **Syntax:**

```
data %>%  
  ggplot(data, aes(x = ...)) +  
  geom_histogram(binwidth = ...,  
                 fill = ...,  
                 color = ...  
                )
```

- **Example:**

```
loan %>%  
  ggplot(aes(x = interest_rate)) +  
  geom_histogram(binwidth = 2.5,  
                 fill = "steelblue",  
                 color = "black"  
                )
```

Splitting Graph into Groups: facet_wrap()



Splitting Graph into Groups: facet_wrap()

- **Syntax:**

```
Your_graph +  
  facet_wrap(~variable,  
             scales = "free_y",  
             nrow = ...)
```

- **Example:**

```
loan %>%  
  ggplot(aes(x = interest_rate)) +  
  geom_histogram(bins = 20) +  
  facet_wrap(~grade,  
             scales = "free_y",  
             nrow = 2)
```


Splitting Graph into Groups: facet_wrap()

- **Example:**

```
loan %>%  
  ggplot(aes(x = interest_rate)) +  
  geom_histogram(bins = 20) +  
  facet_wrap(~grade,  
            scales = "free_y",  
            nrow = 2)
```

- ~grade: Facet by grade. Each unique grade value gets its own panel.
 - Works with multiple variables. e.g. grade + disbursement_method.
- scales = "free_y": Scales setting for x, y axes.
 - "free_y" means x scales vary per panel and y fixed.
 - Other type: fixed (default), free_x, free.
- nrow = 2: Force 2 rows in the panel.
 - ncol works as well.

Barplot: geom_bar()

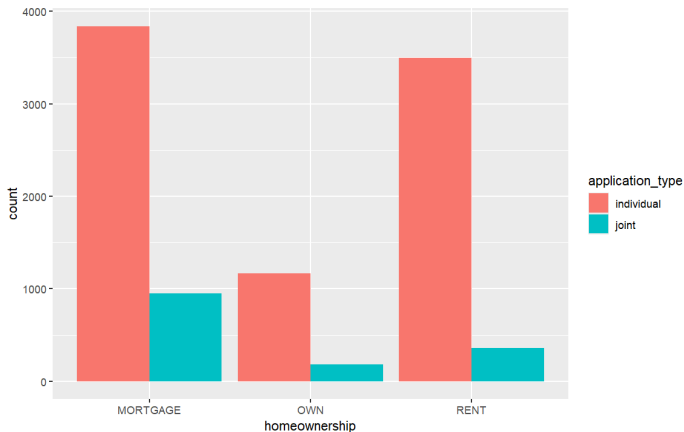


Figure: Bar plot of homeownership by application type.

Barplot: geom_bar()

- **Syntax:**

```
loan %>%  
  ggplot(aes(x = ..., fill = ...)) +  
  geom_bar(position = ...)
```

- **Example:**

```
loan %>%  
  ggplot(aes(x = homeownership, fill = application_type)) +  
  geom_bar(position = "dodge") # stack, fill, ...
```

Add Title and Axis Labels: labs()

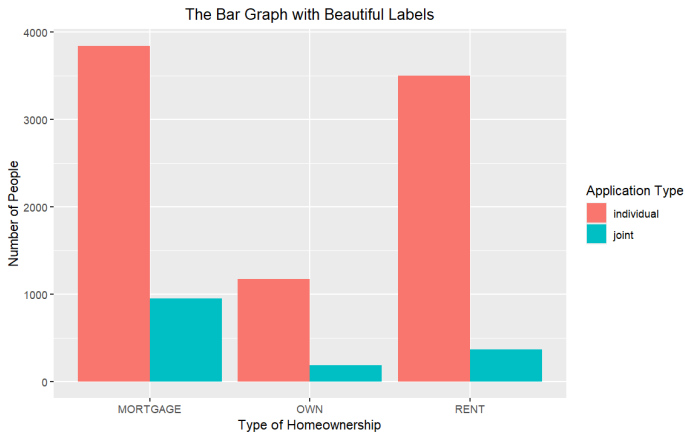


Figure: Same bar plot, **but with title and labels.**

Add Title and Axis Labels: labs()

• Syntax:

```
Your_Plot +  
  labs(x = ..., y = ..., title = ...) +  
  theme(plot.title = element_text(hjust = 0.5))
```

• Example:

```
loan %>%  
  ggplot(aes(x = homeownership, fill = application_type)) +  
  geom_bar(position = "dodge") +  
  labs(  
    x = "Type of Homeownership",  
    y = "Number of People",  
    title = "The Bar Graph with Beautiful Labels",  
    fill = "Application Type"  
  ) +  
  theme(plot.title = element_text(hjust = 0.5))
```

Categorical variables in R: factor()

- A factor stores categorical values with a set of levels.
 - The **order of levels** matters (for plots and for modeling).

- **Syntax:**

```
factor(x, levels = c("level1", "level2", "level3"))  
  
df %>% mutate(cat = factor(cat, levels = c("A", "B", "C")))
```

- **Example:**

```
four_seasons <- c("spring", "summer", "autumn", "winter")  
four_seasons_factor <- factor(four_seasons)  
print(four_seasons_factor)
```

- **Output:**

```
[1] spring summer autumn winter  
Levels: autumn spring summer winter
```

- Default levels sort alphabetically.

Categorical variables in R: factor()

- Use `mutate()` to convert a character column into a factor.
- **Example:**

```
df <- data.frame(cat = c("Large", "Small", "Large"))  
df_factor <- df %>%  
  mutate(cat = factor(cat, levels = c("Small", "Large")))
```

- **Output:**

```
> df_factor  
  cat  
1 Large  
2 Small  
3 Large  
  
> df_factor$cat  
[1] Large Small Large  
Levels: Small Large
```

Categorical variables in R: factor()

- The order of the x-axis follows the factor levels for many ggplot2 functions.
 - Such as `geom_bar()`, `geom_histogram()`, `geom_boxplot()`,...
 - Set the order in `mutate()` before plotting.
- **Example:**

```
df %>%  
  ggplot(aes(x = cat)) +  
  geom_bar()
```

```
df_factor %>%  
  ggplot(aes(x = cat)) +  
  geom_bar()
```


Categorical variables in R: factor()

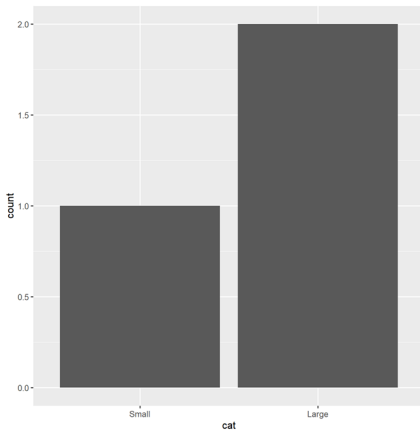
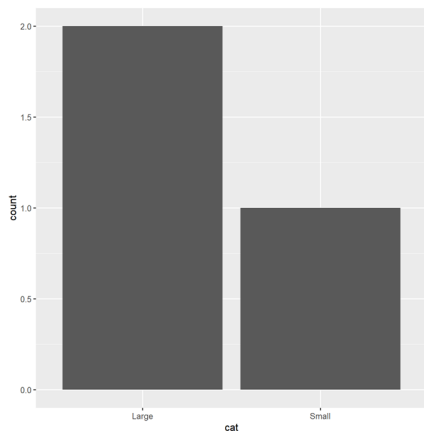


Figure: Left: `df`; Right: `df_factor`

Turn strings into date type: as.Date()

- Sometimes we hope date data can keep the “numeric” properties instead of being a character.
 - as.Date() will turn a specific type of string into Date data.
 - It might show `unknown` when you view in the dataframe though ...
- **Syntax:**

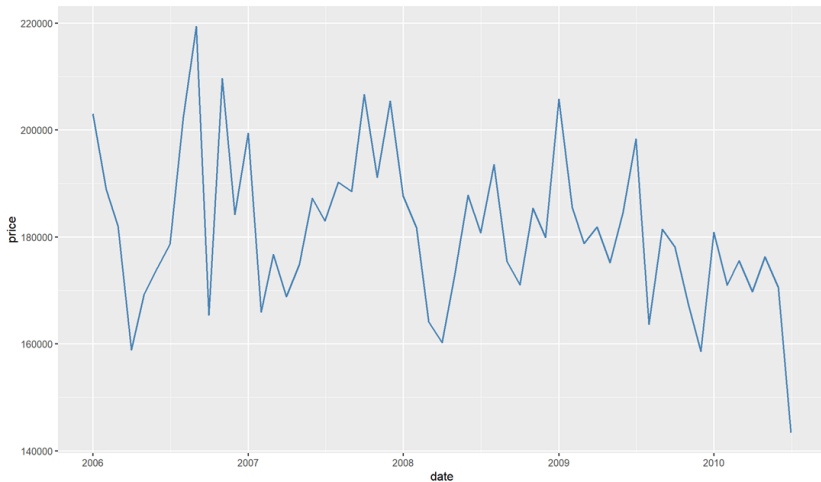
```
as.Date(string)
as.Date(string, format = "%m-%d-%Y") #default %Y-%m-%d
```

- **Example:**

```
as.Date("2025-01-02")
[1] "2025-01-02"

as.Date("01-02-2025", format = "%m-%d-%Y")
[1] "2025-01-02"
```

Draw Line Graph: `geom_line()`



Draw Line Graph: geom_line()

- **Syntax:**

```
data %>%  
  ggplot(aes(x = ..., y = ...)) +  
  geom_line(linewidth = ..., color = ...)
```

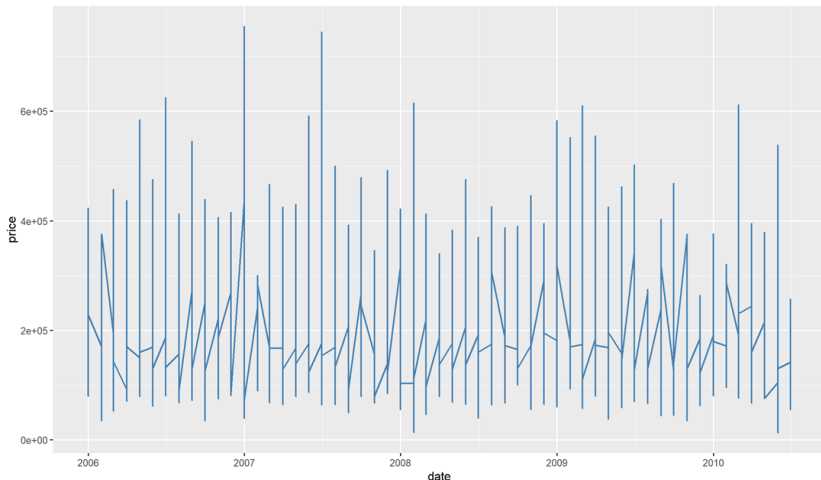
- **Example:**

```
# add "01" to date so it is of the form 2025-06-01  
housing$date <- as.Date(paste(housing$`Yr.Sold`,  
                               housing$`Mo.Sold`,  
                               "01",  
                               sep = "-"))  
  
housing %>%  
  ggplot(aes(x = date, y = price)) +  
  geom_line(linewidth = 0.7, color = "steelblue")
```

- **Is this enough?**

Draw Line Graph: `geom_line()`

- What do we miss here?

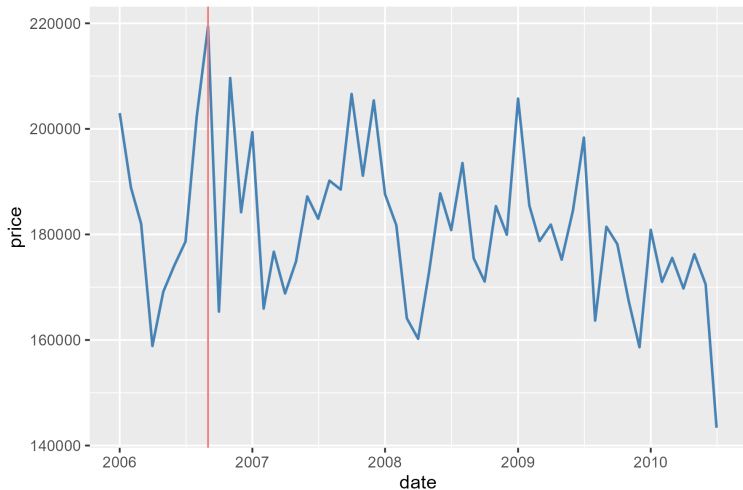


Draw Line Graph: geom_line()

- Each vertical line shows the max and min within that date!
 - What if we want to show the mean of the housing price?
 - Use `stat = "summary", fun = "mean"` in `geom_line()`
- `stat: "summary"` means to reduce to one y per x.
 - Different type of `stat`: `= "count", = "density", ...`
- `fun`: determines how y will be collapsed into a single point.
 - By default, it uses the `"mean"`. Also have: `"median", "sum" ...`
 - `"fun"` has be used together with `stat = "summary"`.
- **Syntax:**

```
housing %>%  
  ggplot(aes(x = date, y = price)) +  
    geom_line(stat = "summary",  
              fun = "mean",  
              linewidth = 0.7,  
              color = "steelblue")
```

Draw Vertical Line: geom_vline()



Draw Vertical Line: geom_vline()

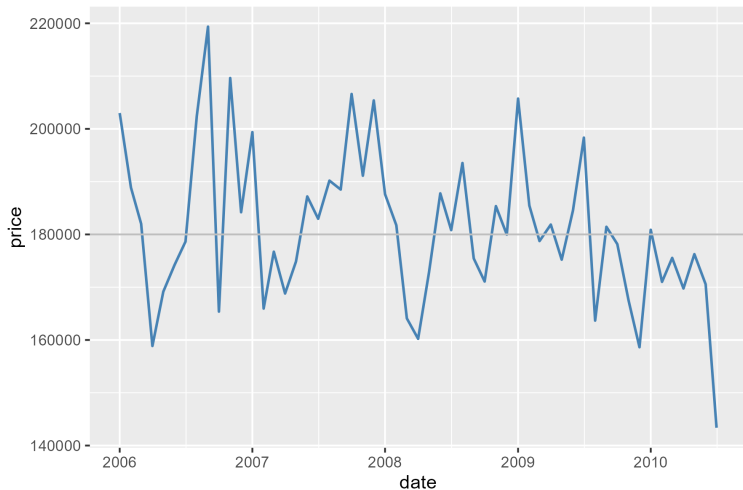
- **Syntax:**

```
Your_graph +  
  geom_vline(xintercept = ..., color = ...)
```

- **Example:**

```
housing %>%  
  ggplot(aes(x = date, y = price)) +  
    geom_line(stat = "summary",  
             fun = "mean",  
             linewidth = 0.7,  
             color = "steelblue") +  
    geom_vline(xintercept = as.Date("2006-09-01"),  
              color = "lightcoral")
```


Draw Horizontal Line: geom_hline()



Draw Horizontal Line: geom_hline()

- **Syntax:**

```
Your_graph +  
  geom_hline(yintercept = ..., color = ...)
```

- **Example:**

```
housing %>%  
  ggplot(aes(x = date, y = price)) +  
    geom_line(stat = "summary",  
             fun = "mean",  
             linewidth = 0.7,  
             color = "steelblue") +  
    geom_hline(yintercept = 180000,  
              color = "gray")
```

Save Your Graph into File: ggsave()

• Syntax:

```
ggsave(filename = ...,  
        plot = ...,  
        path = ...,  
        width = ...,  
        height = ...,  
        dpi = ...)
```

• Example:

```
vline_plot <- ggplot(housing, aes(x = date, y = price)) +  
  geom_line(stat = "summary",  
           fun = "mean",  
           linewidth = 0.7,  
           color = "steelblue") +  
  geom_vline(xintercept = as.Date("2006-09-01"),  
            color = "lightcoral")
```

ggsave(): Which plot is saved?

- By default, ggsave() saves the newest plot showed in the plot window.

```
print(plot_1)
print(plot_2)
ggsave(filename = ...) #It saves plot_2!
```

- Use `plot = ...` to assign which plot you want to save. (Or just put `ggsave()` right after your plot produced.)
- Use `filename = ...` to set the file names and the output type (`.png`, `.jpeg` ...).
- Example:**

```
ggsave(filename = "vline_plot.png",
        plot = vline_plot)
```

ggsave(): Where is my plot saved?

- By default, ggsave() saves your plot to the working directory.
 - It saves to the project folder if you have opened one.
 - But it is better to save in the plots folder!
- Two ways to assign the saved path:
 - Type the path inside filename = ... (absolute or relative).
 - Or use path = ...
- **Example:**

```
ggsave(filename = "C:/Users/you/figures/vline_plot.png")  
ggsave(filename = "figures/vline_plot.png")  
ggsave(filename = "vline_plot.png", path = "figures")
```

ggsave(): How big is my plot?

- By default, ggsave() uses the current plot size in the RStudio Plots window.
 - Which is not good because you may produce difference size of plots every time!
- Use `width = ...` and `height = ...` to control the size.
- Use `dpi = ...` to control the quality of the plot (default = 300).
- **Example:**

```
ggsave(filename = "vline_plot.png",  
       width = 6,  
       height = 4,  
       dpi = 300)
```

ggsave(): Put everything together

- **Example:**

```
ggsave(filename = "vline_plot.png",  
        plot = vline_plot,  
        path = "figures",  
        width = 6,  
        height = 4,  
        dpi = 300)
```